

Progress made during the semester:

- The initial project outline from the first semester was developed further before critical flaws were discovered in the implementation, necessitating the simplification of the codebase.
- SAT and ILP solvers were initially considered as an alternative solution, but were abandoned once more effective algorithmic solutions were found.
- The SAT and ILP formulations proved useful due to shedding light on the fact that multipoles could be generated sequentially by simulating Breadth-First-Search and that recursive code could be implemented more easily for a fixed number of target vertices.
- Research was conducted on whether the simulation of Depth-First-Search or Breadth-First-Search would be more effective at generating $(3,g,s)$ -multipoles. DFS was ultimately ruled out due to requiring more complicated backtracking. Meanwhile, in BFS it was possible to visit each vertex just once per each multipole proposed.
- Research was conducted on algorithms for quickly determining the girth of a graph.
- Research was conducted on canonical graph forms and how isomorphic graphs can be pruned during graph generation or be avoided altogether.

Project structure:

- As the goal of the project is the generation of $(3,g,s)$ -multipoles, the driver code takes g and s as input. Then, it begins generating multipoles, starting from the smallest.
- For each target vertex count, the driver code calls the method `start()` (it also skips some impossible vertex counts).
- `start()` initializes variables. It then calls the method `assign_semiedges()` with initial parameters (or `recursion()`, directly, if s is 0).
- `assign_semiedges()` is called recursively to assign the number of semi-edges. It is optimized so that vertices with semi-edges are a continuous segment at the beginning of the adjacency list, with vertices with more semi-edges being closer to the front. This eliminates all isomorphism introduced by where semi-edges are located.
- `Recursion()`: The basic idea behind the recursive code is to simulate a BFS tree for every possible $(3,g,s)$ -multipole of size N . A non-negligible amount of isomorphic multipoles are still present, but the vast majority of them are never generated.
- During each recursive step, the next free stub is picked. They are always chosen in sequence, from left to right on the adjacency list.

- For the purpose of this explanation, vertex v is considered the parent of vertex u , if there exists an edge $\{v, u\}$ where $v < u$. Similarly, u is considered the child of v .
- Once a stub is picked, the following options are attempted:
 1. Replace the stub with an edge to a potential child that already has a parent or semi-edge.
 2. Replace the stub with an edge to a child that so far only has stubs. (only attempted once, to the vertex with the smallest possible such index – other attempts would only produce isomorphic graphs to the first one).
- Each option, if successful, is followed by checking whether the girth of the graph is still larger or equal to G , calling recursion on the next edge if so, then undoing the option.
- Optimization: children are added in ascending order only.
- Once no free stubs remain, the multipole is a $(3,g,s)$ -multipole and is printed to the console. Then the program backtracks.
- The code yields results at a decent pace up to around 30 vertices, but has potential to be optimized further. Considerable overhead is introduced by larger target girth and isomorphism at higher vertex counts.

Potential optimizations:

- Keeping track of free stubs for faster access.
- Implementing canonicity checking and pruning non-canonical multipoles.
- Optimizing BFS – the same operation is conducted repeatedly on a multipole that changes very little. Perhaps some information gathered could be re-used or the algorithm could be changed to account for this redundancy (eg. remembering minimal distances up to $g - 1$)
- Reducing symmetrical multipoles: If two vertices share one or two parents (they are “siblings”), the second sibling can only have children of higher index than the first.